

Project Blueprint: Autonomous Voice AI Call Center SaaS

Bare-Metal LiveKit Architecture for Sub-500ms Dispatch Execution

LEAD AI ENGINEER

Muhammad Muneeb

DATE

June 2026

ORGANIZATION / ROLE

BSCS / AI Engineer at Devsinn Technologies

TARGET OBJECTIVE

Human-level accuracy, zero-latency
conversational dynamics, < 10 PKR/min cost.

1. Executive Summary

This document outlines the complete, end-to-end engineering roadmap for building a highly optimized, voice-to-voice AI agent platform. The initial target market is the logistics and truck dispatching sector (handling complex constraints, jargon, and noisy environments), with the architecture designed to scale across any BPO or call center domain.

To maximize profit margins and eliminate dependencies on expensive third-party wrappers, this platform utilizes a **top-down engineering methodology**. We bypass standard orchestrators, leveraging a bare-metal LiveKit WebRTC backend, ultra-fast language models (Llama 3 via Groq), and premium streaming STT/TTS (Deepgram). This reduces operating costs from roughly 37 PKR/minute to approximately 7 PKR/minute, enabling massive scalability for international clients.

2. The Architecture Stack

The system is broken down into high-performance, strictly isolated components. We will orchestrate this using Python on the backend, allowing us to maintain stateful WebSockets without intermediary latency.

Layer	Technology Choice	Role & Justification
	Flutter	

Client / Frontend		Powers the WebRTC client application for zero-latency testing, dashboard monitoring, and eventual web/mobile interfaces.
Backend / API	FastAPI + Docker	Serves as the internal mock-TMS (database) layer, executing lightning-fast tool calls and business logic (e.g., checking load rates).
Orchestrator	LiveKit Agents (Python)	The core "brain." Handles real-time audio chunking, interruption (barge-in) routing, and pipeline streaming.
Telephony	SignalWire	Bridges traditional phone networks (SIP trunking) to our WebRTC backend at a fraction of Twilio's cost.
Speech-to-Text	Deepgram Nova-3	Streams partial transcripts instantly. Exceptionally robust against truck engine noise and regional accents.
Intelligence (LLM)	Groq (Llama 3 8B)	LPU-powered inference delivering >800 tokens/sec. Eliminates the "thinking pause" associated with standard LLMs.
Text-to-Speech	Deepgram Aura	Synthesizes natural, highly intelligible voice responses in under 250ms. Highly cost-effective for production scale.

3. Unit Economics & Profitability Strategy

A critical factor in this project is achieving a sustainable, highly profitable operating margin. By structuring the architecture on raw infrastructure rather than SaaS wrappers, we achieve compelling B2B unit economics.

Cost Objective: Operate under \$0.03 (approx. 8 PKR) per minute, allowing us to sell to BPOs at \$0.06 - \$0.08 per minute while undercutting human labor costs (typically \$0.15 - \$0.25+ per minute for US clients).

Component	Cost per Minute (USD)	Cost for 50-Min Call (PKR)
SignalWire (Telephony)	\$0.003	~42 PKR
LiveKit Cloud (Orchestration)	\$0.002	~28 PKR
Deepgram Nova-3 (STT)	\$0.004	~55 PKR
Groq Llama 3 (LLM)	\$0.001	~14 PKR

Deepgram Aura (TTS)	\$0.015	~208 PKR
Total Raw Cost	~\$0.025 / min	~347 PKR / 50 mins

4. Implementation Roadmap (Top-Down Approach)

Following a top-down methodology, we will build the highest-level functioning prototype first to secure stakeholder buy-in, and progressively abstract and optimize the lower-level systems.

Phase 1: The Zero-Cost "Boardroom" Prototype (Weeks 1-2)

Goal: Build a pitch-ready, fully conversational WebRTC agent using free tiers. No telephony costs. Target audience: BPO owners and dispatch investors.

- **Setup LiveKit Cloud:** Register for a free tier account and generate API keys.
- **Python Worker Initialization:** Write the LiveKit AgentSession script integrating Deepgram (using free credits) and Groq (free developer tier).
- **Mock Backend:** Spin up a local FastAPI server containing a dummy JSON database of "Loads", "Drivers", and "Rates". Use Ngrok to expose it.
- **Client App:** Implement a simple Flutter WebRTC interface allowing the user to press a button and talk directly to the Python worker via laptop microphone (zero telecom latency).
- **Milestone:** A sub-500ms conversation where the agent can check dummy load rates and handle interruptions perfectly.

Phase 2: Conversational Dynamics & State Machine (Weeks 3-4)

Goal: Transition from a "chatbot" to an autonomous worker by enforcing strict operational states.

- **Contextual Endpointing:** Integrate Silero VAD locally. Tune it to recognize when a driver is pausing to think versus finishing a sentence.
- **Barge-In Logic:** Program the pipeline to instantly halt TTS playback and truncate the LLM context history the millisecond a human voice interrupts.
- **State Handoffs:** Implement prompt-switching. The agent starts in Greeting_State, verifies ID, and hands off to Negotiation_State or Tool_Execution_State to prevent token bloat and hallucination.

Phase 3: Telephony & Production Infrastructure (Weeks 5-6)

Goal: Connect the system to the real world and ensure 24/7 uptime.

- **SIP Integration:** Purchase a SignalWire phone number and configure the SIP trunk to forward audio via WebSockets to the LiveKit server.
- **Containerization:** Wrap the Python LiveKit worker and FastAPI backend in Docker containers.
- **Cloud Deployment:** Deploy the Docker images to a scalable VPS (e.g., AWS EC2 or DigitalOcean) situated in us-east-1 to physically minimize network latency to US-based telecom providers.

- **Milestone:** A fully functional phone number that anyone can dial into and negotiate a freight rate.

Phase 4: Optimization & Edge Cases (Weeks 7+)

Goal: Polish the AI for the harsh realities of trucking environments.

- **Custom Vocabulary:** Feed Deepgram a specialized dictionary of logistics jargon (BOL, LTL, deadhead, reefer) and specific highway/city names to drastically boost STT accuracy over diesel engine noise.
- **Human-in-the-Loop (Warm Transfer):** Implement a fail-safe trigger. If the sentiment analysis flags the caller as highly agitated, or if a database query fails twice, the system bridges a human dispatcher into the call seamlessly.

5. Technical Blueprint: Core Mechanics

To achieve the desired "human-level" interaction, the system relies on parallel streaming. Below is the conceptual logic for the LiveKit worker:

```
# Conceptual architecture of the LiveKit Worker Pipeline

async def entrypoint(ctx: JobContext):
    # 1. Initialize the models
    stt = deepgram.STT(model="nova-3", language="en-US")
    llm = openai.LLM(model="llama-3-8b-instruct", base_url="https://api.groq.com/openai/v1")
    tts = deepgram.TTS(model="aura-asteria-en")

    # 2. Define the Agent and inject tools (FastAPI backend)
    agent = VoicePipelineAgent(
        vad=silero.VAD.load(),
        stt=stt,
        llm=llm,
        tts=tts,
        chat_ctx=initial_dispatch_context,
        fnc_ctx=DispatchToolRegistry() # Contains check_rate, book_load, etc.
    )

    # 3. Start the continuous listening and reasoning loop
    agent.start(ctx.room)

    # 4. Agent automatically handles barge-in via VAD detection
    # and streams token-by-token directly to Deepgram Aura.
```

6. Next Actions

1. **Environment Setup:** Initialize the Python virtual environment on the Mac Mini, install livekit-agents, fastapi, and uvicorn.
2. **API Procurement:** Generate standard API keys for Deepgram, Groq, and LiveKit Cloud.
3. **Mock Data Construction:** Design the JSON schemas for the TMS load board that the AI will query.

By strictly adhering to this bare-metal, latency-first approach, we bypass the limitations of generic SaaS wrappers. This architecture provides the foundation for a truly disruptive, enterprise-grade AI call center platform.